Training Dynamics of Heavy-Tailed Initilisation in Deep Learning

Yi Hao

March 31, 2026

1 Abstract

2 Methodology

2.1 Dataset and Preprocessing

All experiments are conducted using the MNIST dataset of handwritten digits. The input data consists of 28×28 grayscale images, which are flattened into vectors of size 784. Preprocessing involves a standard normalization transform with a mean of 0.1307 and a standard deviation of 0.3081. We utilize a consistent batch size of 1024 to maintain stable gradient estimates across all initialization regimes.

2.2 Model Architecture

To study the impact of heavy-tailed statistics on signal propagation, we employ a deep Multilayer Perceptron (MLP) with the following specifications:

- **Depth:** 9 hidden layers (10 layers total including the output).
- **Width:** A constant hidden size of 784 neurons per layer to maintain square weight matrices, facilitating Random Matrix Theory (RMT) analysis.
- **Activation:** The hyperbolic tangent (tanh) function is used throughout the hidden layers.
- **Biases:** All bias terms are disabled ($b = 0$) to isolate the dynamical effects of the weight matrix couplings.

2.3 Heavy-Tailed Initialization Strategy

The core of this research involves a parametric sweep over non-Gaussian weight initializations. Weights are sampled from a symmetric Lévy α-stable distribution, $S_\alpha(\sigma)$. We explore a 3×3 grid of hyperparameters to compare different dynamical phases:

- **Stability Parameter (α):** Fixed at $\{1.2, 1.5, 2.0\}$. Here, $\alpha = 2.0$ serves as the Gaussian baseline, while $\alpha < 2.0$ introduces heavy-tails with infinite variance.
- **Scale Parameter (g):** Fixed at $\{0.25, 1.0, 3.0\}$. This parameter dictates the coupling strength $D_w^{1/\alpha}$, allowing us to move the network through ordered, critical, and chaotic regimes.

2.4 Optimization and Training Protocol

The networks are trained for 650 epochs using vanilla Stochastic Gradient Descent (SGD). We set a fixed learning rate of $\eta = 0.001$. Notably, we omit momentum and weight decay; this ensures that any observed structural evolution or "rich learning" behavior is a direct result of the interaction between the heavy-tailed initialization and the fundamental SGD trajectory. All simulations are executed on NVIDIA hardware using the PyTorch framework with a fixed seed ($s = 0$) to ensure exact reproducibility across parameter sweeps.

2.5 Time-Averaged Mean Square Displacement (TAMSD)

To quantify the exploration of the parameter space by the network weights, we compute the Time-Averaged Mean Square Displacement (TAMSD). For a flattened weight vector $\mathbf{w}^l$ at layer l, the TAMSD at a lag time Δt is defined by averaging over the total training time T:

$$\overline{\delta^2}(\Delta t) = \frac{1}{T - \Delta t} \sum_{t=1}^{T-\Delta t} \|\mathbf{w}^l(t + \Delta t) - \mathbf{w}^l(t)\|^2 \tag{1}$$

This approach provides a robust temporal average, smoothing out stochastic fluctuations inherent in individual SGD steps. We utilize a logarithmic sampling of $\Delta t \in \{1, 2, \ldots, 128\}$ to capture dynamics across multiple time scales.

2.6 The Diffusion Exponent (γ)

The nature of weight propagation is characterized by the diffusion exponent γ. We extract this value by performing a linear regression on the log-log TAMSD plot:

$$\log_{10}(\overline{\delta^2}) = \gamma \log_{10}(\Delta t) + C \tag{2}$$

where γ determines the diffusion regime:

- $\gamma \approx 1$**:** Normal Brownian diffusion (typical of standard Gaussian learning).
- $\gamma > 1$**:** Super-diffusion (indicating anomalous transport facilitated by heavy-tailed weights).
- $\gamma < 1$**:** Sub-diffusion (suggesting the network is trapped in local minima or a "lazy" regime).

To ensure the accuracy of the fit and avoid initialization transients, we anchor the regression to the steady-state regime where $\Delta t \geq 4$.

2.7 Cross-Layer Coupling and Correlation Matrices

To investigate the structural redundancy and symmetry breaking mentioned in the primary research aims, we compute cross-layer correlation matrices. We calculate the dependency between the displacement vectors of layer A and layer B at a fixed lag ($\Delta t = 1$). Three metrics are employed to differentiate between linear and non-linear dependencies:

1. **Pearson (r):** Measures the linear relationship between layer trajectories.
2. **Spearman (ρ):** A rank-based correlation that identifies monotonic but potentially non-linear coupling.
3. **Distance Correlation ($dCor$):** A robust metric capable of detecting arbitrary functional dependencies, where $dCor = 0$ implies true independence.

These correlations are visualized as 10×10 heatmaps (representing the 9 hidden layers and 1 classification layer), allowing us to identify if heavy-tailed initializations lead to the "locking" of specific layer dynamics.

2.8 Multi-Scale Weight Snapshots

To investigate the structural evolution of the network throughout the training trajectory, we capture full model state-reflections at 45 non-linearly spaced intervals ($E \in \{0, 1, \ldots, 625\}$). This snapshot strategy enables a detailed "fingerprinting" of every layer without the computational overhead of redundant training runs. We differentiate between *Intrinsic Metrics*, which describe the current state of a weight matrix W, and *Relative Metrics*, which measure the displacement from the initial state W_0.

2.9 Intrinsic Spectral Fingerprinting

For each weight matrix W, we compute a suite of metrics to characterize its spectral and distributional properties:

- **Empirical Tail Index ($\hat{\alpha}$):** We estimate the heavy-tailedness of the weight entries using a Hill Estimator. By selecting the top $k = 1\%$ of absolute weight values, the estimator is defined as:

$$\hat{\alpha} = \left(\frac{1}{k} \sum_{i=1}^{k} \ln \frac{|w_{(i)}|}{|w_{(k)}|} \right)^{-1} \tag{3}$$

  where $w_{(i)}$ represents the i-th order statistic. This allows us to track whether the heavy-tailed nature ($\alpha < 2$) of the initialization is preserved or diminished during training.

- **Spectral Gap:** Defined as the ratio of the two largest singular values σ_1/σ_2. This serves as a proxy for identifying the emergence of discrete spectral outliers, a key prediction of heavy-tailed Random Matrix Theory.

- **Stable Rank:** A robust measure of the matrix's numerical rank, calculated as $\|W\|_F^2/\|W\|_2^2$. This identifies the effective dimensionality of the features learned by each layer.

2.10 Relative Evolutionary Metrics

To quantify the "distance" traveled by each layer in the parameter space, we utilize relative measures compared to the initialization:

- **Relative Frobenius Norm:** Measures the total magnitude of weight change:

$$\|\Delta W\|_{rel} = \frac{\|W - W_0\|_F}{\|W_0\|_F} \tag{4}$$

- **Layer-wise Cosine Distance:** Tracks the rotational evolution of the weight vectors, identifying whether learning involves a simple scaling of initial features or a fundamental reorganization of the principal singular vectors.

2.11 Layer-wise Dynamics and Ratio Analysis

The evolution of these metrics is analyzed across the network's depth to identify the "learning burden" distribution. We specifically track the **Classifier-to-Hidden Ratio**, defined as the ratio of a metric in the final classification layer to the median value across all hidden layers. This ratio identifies phase transitions where the final layer converges faster than the internal feature extractors, a hallmark of the transition between lazy and rich learning regimes.

3 Results

4 Discussion

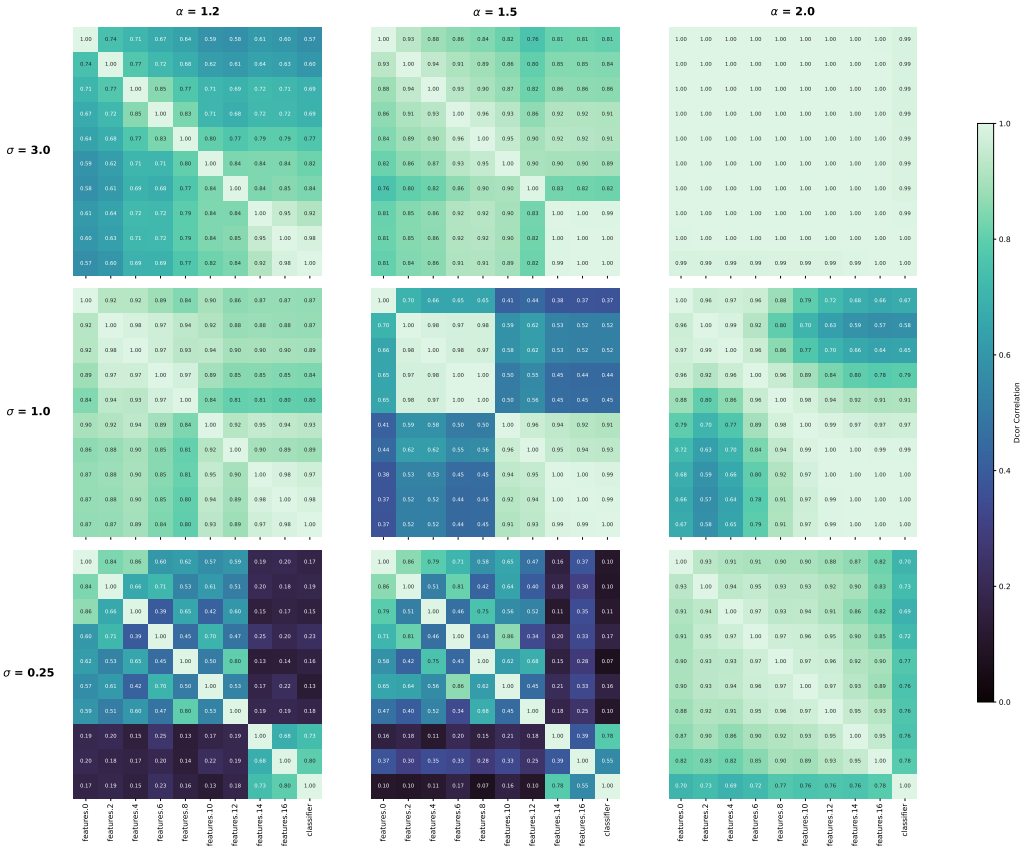

Figure 1: This is a vector-based plot saved as a PDF.